UJIAN TENGAH SEMESTER

(Disusun Untuk Memenuhi Ujian Tengah Semester Desain dan Analisis Algoritma B)



Dosen Pengampu:

Dr. Dyah Sulistyowati Rahayu, S.Kom., M.Kom

Oleh:

Nayunda Krisna

(4524210077)

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS PANCASILA

JAKARTA SELATAN

2025

1. a. Tujuan & kompleksitas Binary Search (Algoritma A) Tujuannya:
Buat nyari data tertentu di dalam list yang udah diurutkan. Misalnya kamu punya data [2, 5, 8, 12, 20], dan mau nyari angka 8.
Nah, Binary Search bakal lihat posisi tengah dulu, terus nentuin mau lanjut nyari ke kiri atau ke kanan.

Waktu (time complexity):
- Terbaik: O(1) → kalau data yang dicari pas di tengah.
- Rata-rata & terburuk: O(log n) → karena tiap langkah, jumlah data yang dicek dibagi dua.

Ruang (space complexity):
- O(1) → karena cuma pakai beberapa variabel aja.

b. Tujuan & kompleksitas Merge Sort (Algoritma B)

Tujuannya:

Buat ngurutin data darikecil kebesar.
Caranya: data dibagi-bagi dulu jadi kecil banget (rekursif), terus nanti digabung lagi sambil diurutin.

Pokoknya kayak motong kue jadi potongan kecil, baru disusun ulang rapi.

Waktu (time complexity):
- Semua kondisi (baik, rata-rata, terburuk): O(n log n)

Ruang (space complexity):
- O(n) → karena dia bikin array tambahan buat proses penggabungan.

c. Mana yang lebih efisien dari sisi waktu?

Dari segi kecepatan:
- Binary Search lebih cepat (O(log n))
- Merge Sort lebih lambat (O(n log n))

Tapi, mereka beda fungsi:
- Binary Search = buat nyari
- Merge Sort = buat ngurutin

Kalau datanya udah urut, Binary Search jelas paling cepat.

Tapi kalau datanya masih acak, ya harus diurutkan dulu pakai Merge Sort, baru bisa nyari pakai Binary Search.

d. Mana yang lebih efisien dari sisi ruang (memori)

Kalau soal hemat memori, jelas:

- Binary Search lebih irit, karena cuma butuh variabel sedikit (O(1)). □ Merge Sort lebih boros, karena dia nyimpen hasil gabungan di array baru (O(n)).

Jadi kalau kamu pengen program yang ringan dan hemat RAM, Binary Search menang di bagian ini.

e. Kapan kondisi ini bisa kebalik (analisis waktu)

Nah ini menarik. Secara teori, Binary Search itu cepat banget.
Tapi itu cuma berlaku kalau datanya udah terurut.

Kalau datanya belum diurutkan, kamu harus sorting dulu (pakai Merge Sort misalnya) baru bisa pakai Binary Search. Artinya, waktu totalnya jadi:

O(n log n) (buat sort) + O(log n) (buat nyari)
Nah, di kondisi kayak gini, analisis “Binary Search lebih cepat” bisa terbalik, karena sorting-nya makan waktu juga. Jadi bisa dibilang, kalau data belum urut, Merge Sort dulu baru Binary Search baru efisien.

2. Langkah 1: Ngurutin Data Transaksi
Karena datanya datang acak banget, hal pertama yang harus dilakukan ya ngurutin semuanya biar rapi.

Dari berbagai algoritma sorting yang ada, pilihan yang cocok buat data gede kayak gini adalah Quick Sort.

Kenapa Quick Sort?
Karena:
Waktunya cepat, rata-rata O(n log n).
Nggak makan banyak memori (cuma O(log n) aja buat stack rekursinya).
Cocok buat data besar kayak 1 juta transaksi.
Di kasus nyata, performanya sering lebih bagus dari Merge Sort yang butuh memori tambahan.

Jadi, tiap hari data transaksi bisa diurutin dulu pakai Quick Sort sebelum dianalisis.

Langkah 2: Ngejawab Pertanyaan (Query)
Setelah semua transaksi rapi dalam array sorted_transactions, semua pertanyaan bisa dijawab super cepat tanpa ngulang sorting lagi.
Contohnya:

“Berapa transaksi antara 1 juta – 5 juta?”

Gunakan Binary Search dua kali:
Sekali buat cari batas bawah (>= 1 juta)
Sekali lagi buat batas atas (<= 5 juta)
Selisih dua posisi itu = jumlah transaksi di rentang itu.

Waktu yang dibutuhin cuma O(log n) per pertanyaan — cepat banget meski query-nya 100 ribu kali.

“Nilai median transaksi?”
Karena datanya udah urut, tinggal ambil elemen tengah:
sorted_transactions[n // 2]
Cuma butuh waktu O(1).

“Transaksi terbesar ke-10?” Langsung
ambil dari belakang:
sorted_transactions[-10]
Juga butuh waktu O(1) aja.

Kesimpulan:
Dengan desain ini:
Sorting cuma dilakukan sekali per hari (pakai Quick Sort).
Setelah itu, semua analisis bisa dilakukan cepat banget.
Query sederhana kayak median, nilai maksimum/minimum, dan range transaksi semuanya bisa dijawab tanpa repot.